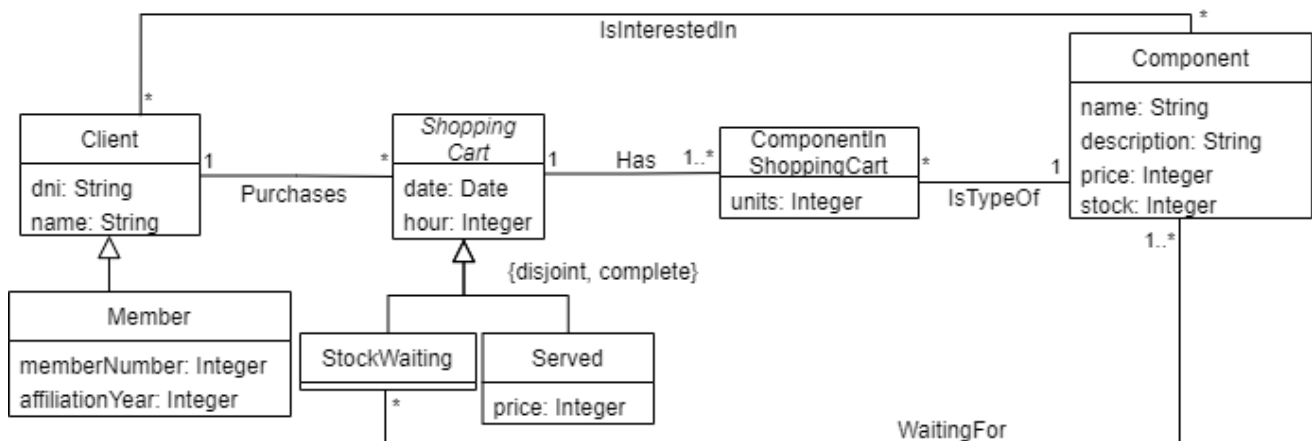


Análisis y Diseño con Patrones

Práctica 2: Patrones de Diseño y de Asignación de Responsabilidades

En la Práctica 1, una tienda que vende componentes hardware (ratones, pantallas, etc...) por internet nos ha pedido que le diseñemos una parte de un sistema software para gestionar las ventas de estos componentes. Ahora nos pide que añadamos nuevas funcionalidades en el software.

A continuación disponéis del diagrama estático de análisis de la Práctica 1 del que partiremos:



Claves de las clases:

- *Client*: `dni`
- *Member*: `memberNumber`
- *ShoppingCart*: `date` + `hour`
- *ComponentInShoppingCart*: `date` + `hour` (*ShoppingCart*) + `name` (*Component*)
- *Component*: `name`

Restricciones de integridad:

- El stock y precio de un componente es superior a 0.
- El número de unidades de un componente en una compra es superior a 0.
- Los componentes en espera de stock de una compra tienen que ser componentes de aquella compra.

- El precio de una compra servida es la suma de los precios de sus componentes multiplicado por la cantidad de los componentes. Si la compra la hace un socio, el importe se decrementará en un 10%.

Pregunta 1 (20%)

Enunciado

En nuestra tienda en línea, nos gustaría implementar un mecanismo que nos permita enviar correos electrónicos tanto a usuarios registrados como no registrados que quieran ser notificados cuando vuelva a estar disponible un componente concreto que actualmente se encuentra agotado.

Para habilitar esta funcionalidad, hemos introducido una nueva clase llamada *UnregisteredUser*, que almacena la dirección de correo electrónico del usuario y también una función global *sendMail(email: String, texto:String)* que envía un correo electrónico a la dirección de correo electrónico especificada.

Para conseguirlo, pretendemos diseñar un sistema desacoplado con bajo acoplamiento entre *UnregisteredUser* y *Component* que envíe correos electrónicos a los usuarios no registrados siempre que el stock de un componente cambia de cero a un número positivo.

La operación *Component::stockArrive(quantity:Integer)* tendría que aumentar el stock del componente e iniciar el envío de correos electrónicos.

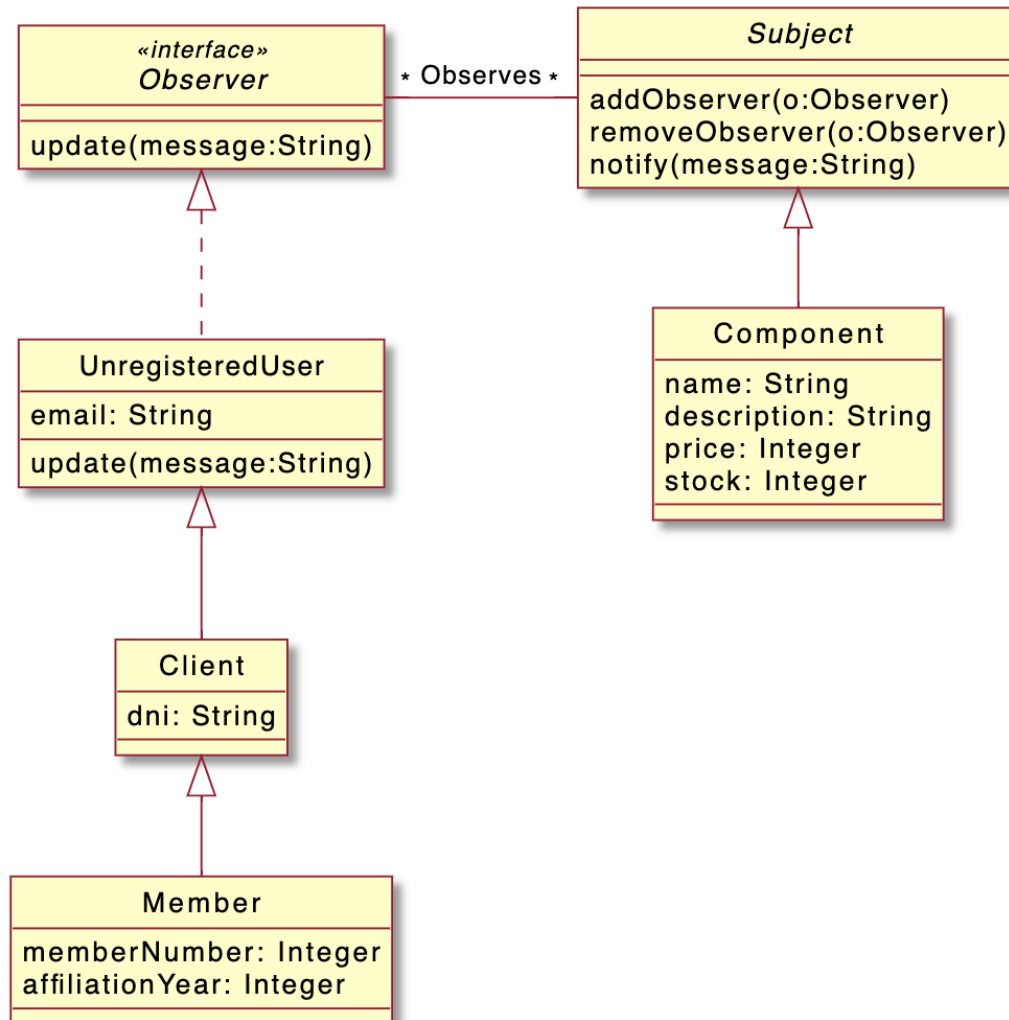
Se pide:

- a) ¿Qué patrón de diseño usarías para diseñar la operación *Component::stockArrive(quantity:Integer)*?
- b) Muestra la parte del diagrama de clases que se ve modificada como resultado de la aplicación del patrón/ns utilizado/s.
- c) Muestra el diagrama de secuencia o el pseudocódigo de la operación *Component::stockArrive(quantity:Integer)*.

Solución

- a) Para diseñar la operación *Component::stockArrive(quantity: Integer)* con un bajo acoplamiento se puede usar el patrón de diseño **Observer**. El patrón de diseño Observer permite establecer una dependencia one-to-many entre los objetos de una clase (subject) con muchos observadores (en este caso los usuarios no registrados que quieren ser notificados) de forma que los observadores son notificados automáticamente cuando el Subject tiene algún cambio (el stock de un componente).

b) El diagrama de clases:



c) El pseudocódigo:

```

public interface Observer {
    public void update();
}

public class Subject {
    private List<Observer> observers;

    public void addObserver(Observer o) {
        this.observers.add(o);
    }

    public void removeObserver(Observer o) {
    }
}
    
```

```
        this.observers.remove(o);
    }

    public void notify(String message) {
        for (Observer o : this.observers) {
            o.update(message);
        }
    }
}

public class Component implements Subject {
    private String name;
    private String description;
    private Integer price;
    private Integer stock;

    public stockArrive(quantity: Integer) {
        if (stock == 0) {
            this.notify("Component "+name+" is available again");
        }
        this.stock += quantity;
    }
}

public class UnregisteredUser implements Observer {
    private String email;

    public void update(String message) {
        sendMail(email, message);
    }
}

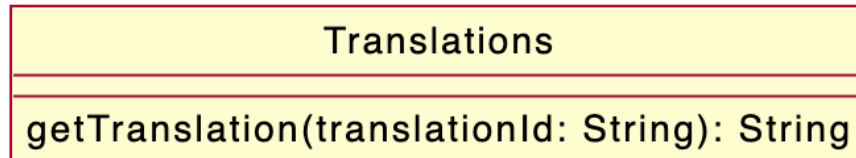
public class Client implements Observer {
    private String dni;
}
```

Pregunta 2 (20%)

Enunciado

El sistema que estamos diseñando ofrece apoyo para varios idiomas. Para conseguirlo, disponemos de una clase de traducciones, que ha sido diseñada para obtener traducciones basadas en un identificador de traducción y el idioma actual del sistema.

Aquí tenéis un diagrama de clases de la clase de traducciones:



Esta clase se instancia siempre que necesitamos mostrar un mensaje localizado al usuario, tal como se muestra al ejemplo siguiente:

```

class ClientsController {

    void newClient(dni: String, name: String): Client {
        Client c = new Client(dni, name);
        Translations t = new Translations();
        view.setMessage(t.getTranslation(WELCOME_TO_SHOP_MESSAGE_ID));
        return c;
    }

}
  
```

Aun así, estamos teniendo algunas implicaciones de rendimiento, relacionadas con la creación y destrucción de múltiples objetos de *Translations*, puesto que el uso de traducciones es generalizado a todo el sistema.

Para solucionar este problema, tenemos que encontrar una solución que permita el uso eficiente del objeto *Translations* a todo el sistema, sin necesidad de crear múltiples instancias o pasarla como parámetro con frecuencia. Esta solución mejorará el mantenimiento y la escalabilidad del sistema.

Se pide:

- ¿Qué patrón de diseño usarías para diseñar la solución?
- Muestra la parte del diagrama de clases que se ve modificada como resultado de la aplicación del patrón/ns utilizado/s.

- c) Muestra el diagrama de secuencia o el pseudocódigo de la operación `ClientsController::newClient(dni: String, name: String): Client` y de todas las operaciones invocadas desde la operación con la aplicación del patrón que has identificado en el apartado.

Solución

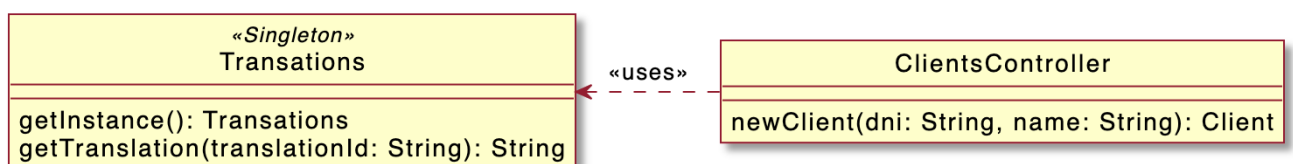
- a) Para diseñar una solución que permita el uso eficiente del objeto *Translations* sin crear múltiples instancias ni pasarlo como parámetro con frecuencia, podéis utilizar el patrón de diseño **Singleton**. El patrón Singleton garantiza que solo se crea una instancia de una clase y proporciona un punto de acceso global a esta instancia. En este caso, la clase *Translations* se puede implementar como Singleton para proporcionar una única instancia compartida a todo el sistema.

A continuación se explica cómo podéis implementar el patrón Singleton para la clase *Translations*:

- Modificamos la clase *Translations* para que su constructor sea privado. Esto evita que la clase se instancie directamente desde el exterior.
- Añadimos un método estático denominado `getInstance()` en la clase *Translations*. Este método se encargará de crear y devolver la única instancia de la clase *Translations*.
- Dentro del método `getInstance()`, comprobamos si ya existe una instancia de *Translations*. Si lo hace, devolvéis esta instancia. Si no, creáis una instancia nueva y guardadla en una variable estática privada.
- Modificamos el método `newClient()` en la clase *ClientsController* para recuperar la instancia de *Translations* intermediando `Translations.getInstance()` en lugar de crear una instancia nueva cada vez.

Al implementar el patrón Singleton, os aseguráis que solo hay una instancia de la clase *Translations* a todo el sistema. Esto elimina las implicaciones de rendimiento de crear y destruir múltiples instancias.

- b) El diagrama de clases:



- c) El pseudocódigo:

```
class Translations {
```

```
private static Translations instance;

private Translations() {}

public static Translations getInstance() {
    if (instance == null) {
        instance = new Translations();
    }
    return instance;
}

public String getTranslation(String translationId) {
    ...
}

}

class ClientsController {

    ...

    void newClient(String dni, String name) {
        Client c = new Client(dni, name);
        Translations t = Translations.getInstance();
        view.setMessage(t.getTranslation(WELCOME_TO_SHOP_MESSAGE_ID));
        return c;
    }
}
```

Pregunta 3 (20%)

Enunciado

Queremos implementar un nuevo mecanismo de descuento en la tienda en línea. Se pueden aplicar diferentes tipos de descuentos a un carrito de la compra, pero solo se puede aplicar un descuento a la vez. Los descuentos se aplicarán cuando se ejecute el método

ShoppingCart::getPriceWithDiscount() y se devolverá el precio final rebajado. Ya tenemos implementado el método *ShoppingCart::getPrice()*, que calcula el precio de los productos del carrito sin ningún descuento.

El primer tipo de descuento que se puede aplicar es un descuento porcentual directo al precio de compra.

El segundo tipo de descuento reduce un importe fijo, por ejemplo, 20 €, si nuestra compra llega a un valor mínimo, como por ejemplo 100 €.

Se prevé que se puedan introducir más tipos de descuentos en el futuro, y nuestro sistema tendría que poder añadirlos fácilmente.

Además, nos tenemos que esforzar para minimizar el acoplamiento entre la parte de la funcionalidad de descuento y otras partes del sistema. De este modo, aseguramos que los cambios o las actualizaciones realizadas a la funcionalidad de descuento no tengan un impacto significativo en otras partes del sistema.

Se pide:

- ¿Qué patrón de diseño usarías para diseñar esta funcionalidad?
- Haz el diagrama de clases con las operaciones resultantes después de aplicar el patrón (podéis proporcionar solo la parte de aplicación del patrón).
- Muestra el diagrama de secuencia o el pseudocódigo de la operación *ShoppingCart::getPriceWithDiscount(): Integer* y de todas las operaciones invocadas desde la operación.

Solución

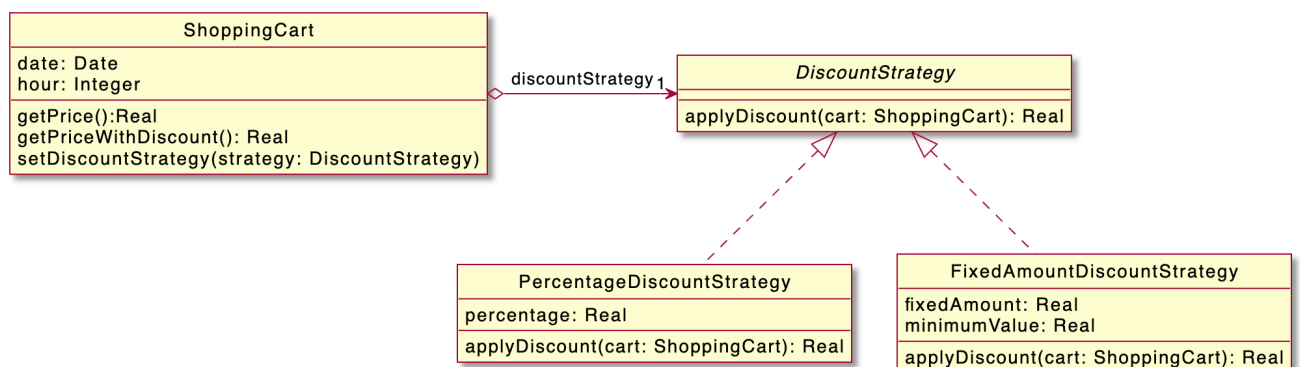
- Para diseñar el mecanismo de descuento de una manera que minimice el acoplamiento y permita adaptar fácilmente los futuros tipos de descuento, podéis utilizar el patrón de diseño **Strategy**. El patrón estrategia nos permite definir una familia de algoritmos (tipos de descuento) encapsulados en clases separadas y hacerlos intercambiables en tiempos de ejecución. Cada tipo de descuento se implementa como una clase de estrategia independiente y la clase *ShoppingCart* tiene una referencia a la estrategia de descuento aplicada actualmente.

A continuación se explica cómo podéis implementar el patrón de estrategia para el mecanismo de descuento:

- Creamos una interfaz llamada *DiscountStrategy*, que declara un método como *applyDiscount(price: Double): Double*. Este método coge el precio original de la carretilla de la compra y devuelve el precio con descuento.
- Implementamos clases concretas de estrategia de descuento que implementen la interfaz *DiscountStrategy*. Por ejemplo, podéis crear clases como *PercentageDiscountStrategy* y *FixedAmountDiscountStrategy*, cada una con su propia lógica para aplicar el tipo de descuento correspondiente.
- Modificamos la clase *ShoppingCart* para incluir un campo del tipo *DiscountStrategy*, denominamos *discountStrategy*. Este campo incluirá la estrategia de descuento aplicada actualmente.
- Añadimos un método setter en la clase *ShoppingCart*, como por ejemplo *setDiscountStrategy(strategy: DiscountStrategy)*, para definir la estrategia de descuento que se tiene que aplicar.
- Modificamos el método *getPriceWithDiscount()* en la clase *ShoppingCart* para gritar el método *applyDiscount(price: Double)* al objeto *discountStrategy*, pasando la carretilla. Este método devolverá el precio final rebajado.

Este patrón de diseño minimiza el acoplamiento porque la clase *ShoppingCart* no necesita conocer los detalles específicos de cada tipo de descuento. Solo interactúa con la interfaz de *DiscountStrategy*, lo cual permite añadir o modificar fácilmente los tipos de descuento sin afectar otras partes del sistema.

b) El diagrama de clases:



c) El pseudocódigo:

```

class ShoppingCart {
    ...

    private DiscountStrategy discountStrategy;

    public void setDiscountStrategy(DiscountStrategy discountStrategy) {

```

```
        this.discountStrategy = discountStrategy;
    }

    public Real getPriceWithDiscount() {
        return getPrice() - discountStrategy.applyDiscount(this);
    }
}

public interface DiscountStrategy {
    Real applyDiscount(ShoppingCart cart);
}

public class PercentageDiscountStrategy implements DiscountStrategy {
    private Real percentage;

    public PercentageDiscountStrategy(Real percentage) {
        this.percentage = percentage;
    }

    public Real applyDiscount(ShoppingCart cart) {
        return cart.getPrice() * percentage;
    }
}

public class FixedAmountDiscountStrategy implements DiscountStrategy {
    private Real fixedAmount;
    private Real minimumValue;

    public FixedAmountDiscountStrategy(Real fixedAmount, Real
minimumValue) {
        this.fixedAmount = fixedAmount;
        this.minimumValue = minimumValue;
    }

    public Real applyDiscount(ShoppingCart cart) {
        if (cart.getPrice() >= minimumValue) {
            return fixedAmount;
        }
        return 0;
    }
}
```

Pregunta 4 (20%)

Enunciado

Queremos implementar una nueva funcionalidad que proporciona todos los componentes que un cliente está esperando en las diferentes compras que ha hecho y todavía están pendientes del stock.

El método *Client::getAllWaitingForComponents():List<Tuple<String,Integer>>* tiene que devolver una lista que contenga tuplas con el nombre del componente y el número de piezas que el usuario está esperando. Tened en cuenta que el número de piezas de componentes tiene que ser el total acumulado de todas las compras del usuario.

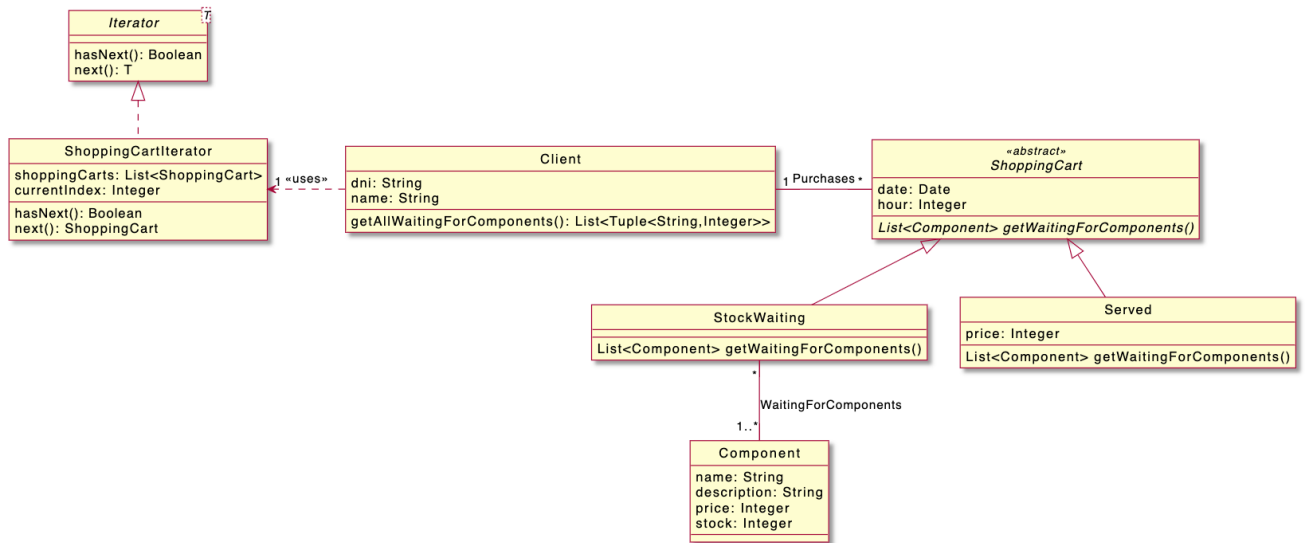
Es importante mantener un bajo acoplamiento mientras se implementa esta operación, como es habitual. Esto garantizará que los cambios o actualizaciones de esta funcionalidad no tengan un impacto significativo en otras partes del sistema.

Se pide:

- ¿Qué patrones de diseño usarías para diseñar la nueva funcionalidad?
- Haz el diagrama de clases con las operaciones resultantes después de aplicar los patrones.
- Muestra el diagrama de secuencia o el pseudocódigo de la operación *Client::getAllWaitingForComponents():List<Tuple<String,Integer>>* y de todas las operaciones invocadas desde la operación.

Solución

- El patrón **Iterator** se puede usar para desacoplar la lógica transversal de las clases del carrito de compras. La clase *ShoppingCartIterator* encapsula el proceso de iteración y proporciona una forma estandarizada de iterar sobre los componentes de un carrito de compras. Este desacoplamiento permite que la clase *Client* itere sobre los componentes sin estar estrechamente acoplada a la estructura interna y los detalles de implementación de las clases del carrito de compras.
- El diagrama de clases:



c) El pseudocódigo:

```

class Client {
    public List<String,Integer> getAllWaitingForComponents() {
        List<String,Integer> waitingComponents = new ArrayList<>();
        ShoppingCartIterator iterator = new ShoppingCartIterator(purchases);

        while (iterator.hasNext()) {
            ShoppingCart shoppingCart = iterator.next();
            List<Component> components =
shoppingCart.getWaitingForComponents();

            for (Component component : components) {
                int i = 0;
                boolean found = false;
                while ((i < waitingComponents.length) && (!found)) {
                    if (waitingComponents[i].get(0).getName() ==
                        component.getName()) {
                        waitingComponents[i].get(1)++;
                        found =true; }
                    else {
                        i++;
                    }
                }
                if (!found) {
                    waitingComponents[waitingComponents.length].add(component.ge
tName(),1);
                }
            }
        }
    }
}
    
```

```
        }

        return waitingComponents;
    }
}

class ShoppingCartIterator implements Iterator<ShoppingCart> {
    private List<ShoppingCart> shoppingCarts;
    private int currentIndex;

    public ShoppingCartIterator(List<ShoppingCart> shoppingCarts) {
        this.shoppingCarts = shoppingCarts;
        currentIndex = 0;
    }

    public boolean hasNext() {
        return currentIndex < shoppingCarts.size();
    }

    public ShoppingCart next() {
        ShoppingCart shoppingCart = shoppingCarts.get(currentIndex);
        currentIndex++;
        return shoppingCart;
    }
}

abstract class ShoppingCart {
    private List<Component> waitingForComponents;

    public abstract List<Component> getWaitingForComponents();
}

class Served extends ShoppingCart {
    ...
    public List<Component> getWaitingForComponents() {
        return new ArrayList<>();
    }
}

class StockWaiting extends ShoppingCart {
    ...
    public List<Component> getWaitingForComponents() {
        return waitingForComponents;
    }
}
```

```
}
```

Pregunta 5 (20%)

Enunciado

Hemos ampliado nuestro negocio y ya no vendemos solo componentes de hardware; también hemos incluido otros productos de terceros.

Para dar cabida a la creciente gama de productos, hemos implementado categorías. Aun así, diferenciamos entre categorías para nuestros componentes internos y categorías para los componentes de proveedores externos. Tenemos una clase abstracta llamada *Category*, *InternalCategory* para nuestros propios componentes y *ExternalCategory* para los componentes disponibles en las tiendas externas. La interfaz *ICategory* proporciona el conjunto de operaciones que tiene que implementar una categoría.

Para mejorar la experiencia de usuario, queremos implementar una funcionalidad de búsqueda donde los usuarios puedan buscar componentes introduciendo algún texto que se encuentre en el nombre o descripción del componente.

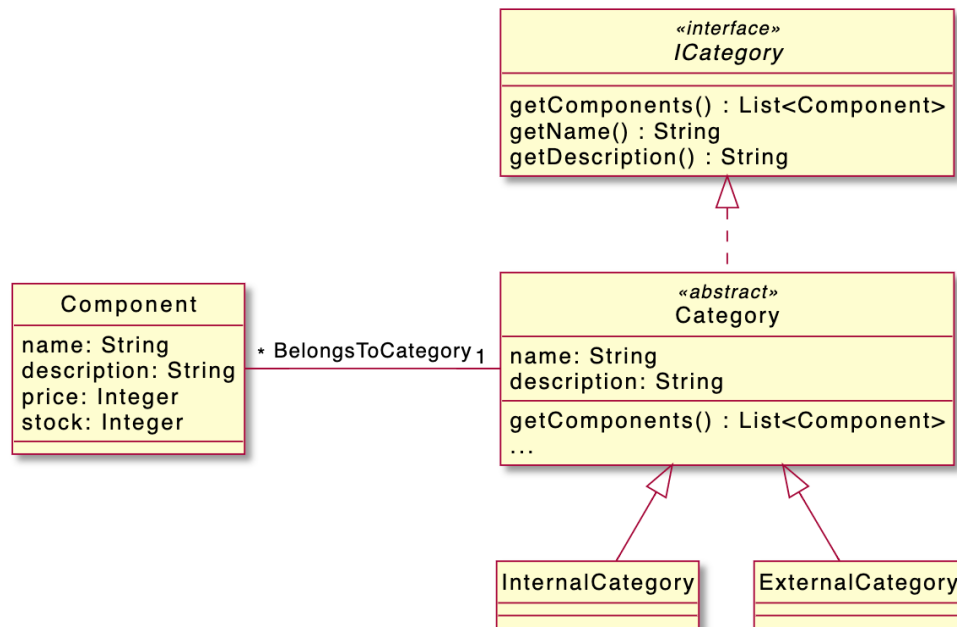
Para conseguirlo, se nos ha proporcionado la interfaz *Searchable*, que contiene la operación *search(String text)*.

```
public interface Searchable {  
    List<Component> search(String text);  
}
```

Queremos hacer búsquedas a partir de las categorías existentes, de forma que cuando un usuario introduzca un texto, el método *search(text)* devuelva una lista de todos los componentes de la categoría que coinciden con el texto. Un componente coincide con el texto si su nombre o descripción contiene el texto.

Como la funcionalidad de búsqueda puede ampliarse en el futuro y ser más compleja, tenemos que implementarla fuera de las clases de categoría y sin modificar esta clase.

A continuació disponéis del diagrama de classes del que partiremos:



Se pide:

- ¿Qué patrón de diseño usarías para diseñar esta funcionalidad?
- Haz el diagrama de clases con las operaciones resultantes después de aplicar el patrón (podéis proporcionar solo la parte de aplicación del patrón).
- Muestra el diagrama de secuencia o el pseudocódigo de las tres operaciones descritas

Solución

- Para implementar la funcionalidad de busca para todas las categorías de clase sin modificarlas y fuera de las clases de categoría, podéis utilizar el patrón de diseño de **Decorator**.

El patrón Decorator os permite añadir nuevas funcionalidades a un objeto de forma dinámica con un comportamiento adicional. En este caso, podéis crear una clase *SearchableCategory* que implemente la interfaz *Searchable* y decore las clases de categoría existentes.

A continuación se explica cómo podéis implementar el patrón Decorator para diseñar la funcionalidad de busca:

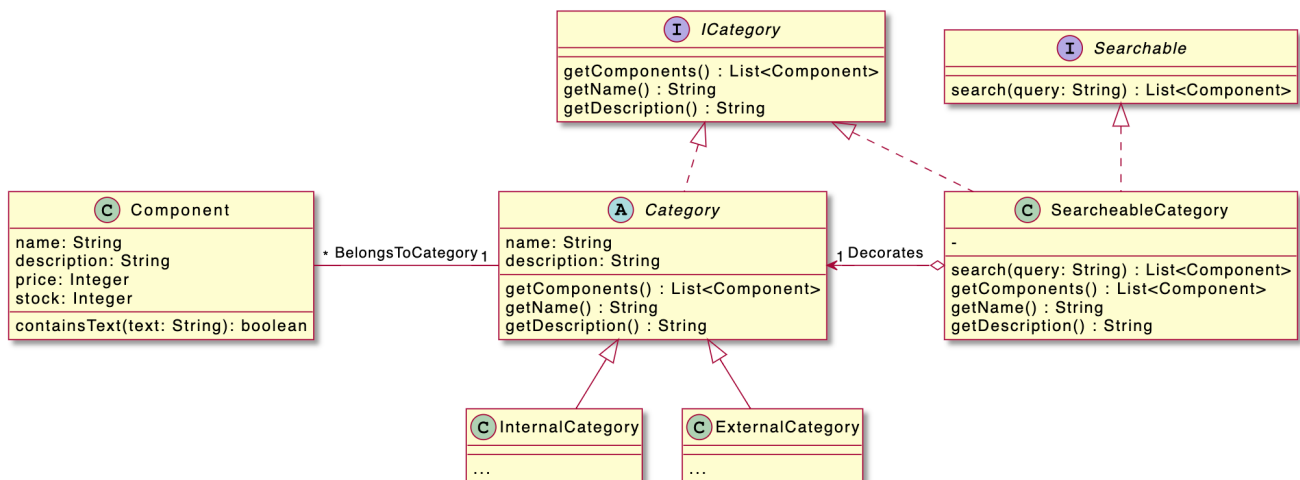
- Creamos una clase llamada *SearchableCategory* que implementa la interfaz *Searchable*. Esta clase actuará como decorador de las clases de la categoría.
- A *SearchableCategory* añadimos una referencia a una instancia de la clase de categoría. Esta referencia se utilizará para delegar los métodos al objeto de categoría envuelto.
- Implementamos el método *search(String text)* a *SearchableCategory*, llamando a los métodos necesarios al objeto de categoría delegado, en este caso *Category::getComponents()*.

Al código de cliente, cuando necesitamos utilizar la funcionalidad de busca, creamos una instancia de *SearchableCategory* y pasamos el objeto de categoría original. Esto decorará el objeto de categoría con la funcionalidad de busca.

Mediante el patrón Decorator, podemos añadir la funcionalidad de busca en las clases de categoría sin modificarlas directamente. La clase decoradora proporciona el comportamiento de busca mientras mantiene intactas los objetos de la categoría original.

Este diseño permite una fácil extensibilidad y flexibilidad. Si necesitáis introducir funcionalidades adicionales o modificar el comportamiento de busca en el futuro, podéis crear nuevas clases de decorador o modificar la clase de decorador existente sin afectar las clases de categoría o el código existente.

b) El diagrama de clases:



c) El pseudocódigo:

```

class SearchableCategory implements ICategory implements Searchable {

    private ICategory decorates;

    public SearchableCategory(ICategory category) {

```



```
        decorates = category;
    }

    public String getName() {
        return decorates.getName();
    }

    public String getDescription() {
        return decorates.getDescription();
    }

    public List<Component> getComponents() {
        return decorates.getComponents();
    }

    List<Component> search(String text) {
        List<Component> result = new List<Component>();
        for (Component c : getComponents()) {
            if (c.containsText(text) {
                result.add(c);
            }
        }
        return result;
    }
}

class Component {

    ...

    boolean containsText(String text) {
        return this.name.contains(text) || this.description.contains(text);
    }
}
```